

Node.js Production Debugging

Notebook Cheat Sheet + Publicly Reported Interview Question Bank

Prepared from the uploaded Node.js debugging notes plus publicly indexed interview experiences/posts. "Reported" means the question/theme was published by a candidate/interview source; "Practice" is a derived scenario for preparation and is not claimed to have been asked by a specific company.

PART 1 — CHEAT SHEET FOR YOUR NOTEBOOK

1. FIRST RULE — BASELINE BEFORE TOUCHING

- Do not restart first. Capture which instance is bad, CPU, memory/RSS, p95/p99 latency, event-loop delay and recent deploys.
- One instance degrading → suspect instance-local leak/state. All instances rising together → first investigate a shared code path, traffic pattern or dependency.
- Always compare before/after metrics so the fix is measurable.

2. CPU PROFILING / FLAME GRAPHS

- CPU profiling answers: which functions are consuming CPU samples?
- Flame graph X-axis = frequency across samples, NOT duration of one call.
- Y-axis = call-stack depth; root is at the bottom, leaf frames at the top.
- Wide TOP plateau → that leaf function itself is expensive → optimize it.
- Wide MIDDLE frame with several children above → parent width is the total of child work; inspect the children.
- Wide `v8::internal::Heap::*` frames → GC is consuming CPU because allocation/heap pressure is high; reduce allocations rather than optimizing an arbitrary app frame.
- Production spike now → do not start a fresh profiling process; attach to the running process / inspector where possible.

3. MEMORY LEAKS / HEAP SNAPSHOTS

- Leak = an object is no longer needed but is still reachable, so GC cannot reclaim it.
- Take two heap snapshots under comparable load, a few minutes apart; compare by Delta.
- Large one-directional growth in one object type is suspicious; small Array/String movement can be normal.
- LCC mnemonic: Listener accumulation, Closure retention, Cache without eviction.
- Listener → `removeListener()/off()`, or use `once()` when appropriate.
- Closure → retain only the fields the callback really needs.
- Cache → enforce a maximum size and/or LRU/TTL eviction.
- Do not treat periodic restarts as the root-cause fix.

4. EVENT LOOP / LATENCY

- High event-loop delay can occur even when total CPU looks normal: one synchronous/CPU-heavy operation blocks the single JS thread.
- Use `perf_hooks.monitorEventLoopDelay()` and inspect p50/p95/p99/max rather than relying only on averages.
- Common blockers: large `JSON.parse/stringify`, synchronous filesystem work, CPU-heavy loops, expensive regex, unbatched crypto.
- CPU hot + latency hot → profile CPU and correlate with loop delay.
- CPU normal + latency hot across endpoints → strongly investigate event-loop blocking, downstream waits, pool exhaustion and queues.

5. LIVE DEBUGGING / CRASH ANALYSIS

- V8 inspector can be enabled on a running process; tunnel access privately. Never expose the inspector publicly because it is not an authenticated public debugging interface.
- For post-mortem OOM/crash analysis, enable core dumps before the incident and analyze the core with llnode.
- Continuous profiling is useful because it preserves historical windows and lets you compare before/after deploys.

6. PRODUCTION GUARDRAILS

- Set a deliberate V8 heap limit appropriate to the container rather than assuming the default is correct.
- Store large heap snapshots off-instance when practical, especially when disk/memory pressure is already high.
- Overlay deploy markers on profiling/monitoring dashboards.
- Profile under realistic production-like load; quiet environments can hide JIT and allocation behavior.
- Keep a concise incident trail: symptom → evidence → hypothesis → test → root cause → fix → verification.

7. 45-SECOND SENIOR ANSWER FRAMEWORK

- Problem: state the production symptom and blast radius.
- Investigation: name the metrics/tools you would check first.
- Evidence: explain what would confirm or reject your hypothesis.
- Fix: make the smallest safe production change.
- Safety: mention rollback/feature flag/limited rollout where relevant.
- Verification: prove the change using the same metric that exposed the problem.
- Trade-off: mention what the fix costs or what you would monitor next.

PART 2 — PUBLICLY REPORTED INTERVIEW QUESTIONS

These are paraphrased from publicly accessible interview reports/posts to keep the document concise and avoid presenting copied text as original. They are grouped by the source that reported them.

Glassdoor — Laerdal Medical Corp. Bengaluru, Senior MERN Stack Developer, Mar 2026

1. Explain a system you built, a production issue you faced, and how you would scale it.
2. Which microservices did you use in your system?
3. How would you implement rate limiting?
4. How would you design health checks/monitoring?
5. Explain the Node.js event loop.
6. What can happen if `process.nextTick()` is used heavily?
7. How would you implement graceful shutdown in Node.js?
8. How would you detect a memory leak?
9. What is `process.exit` and when would you use it?
10. How would you improve API performance?
11. How would you handle a bulk response?
12. How would you approach security for the API?

LinkedIn — Pravin Pagare, Node.js interview experience, Sep 2026

1. Explain your backend architecture.
2. What production challenges have you faced?
3. How do you handle scalability and high traffic?
4. What is the Node.js event loop?
5. What is the difference between synchronous and asynchronous code?
6. Explain Promises and `async/await`.
7. What is the difference between `process.nextTick()` and `setImmediate()`?
8. How does Node.js handle multiple requests?
9. What are Streams and Buffers?
10. How do you handle logging, monitoring and exception handling in production?
11. How do you safely deploy without downtime?
12. How do you investigate a sudden spike in API latency?
13. What happens when one microservice is unavailable?

LinkedIn — Ankita Budhia, Full Stack interview experience, Sep 2026

1. How would you prevent memory leaks in a long-running Node.js application?
2. What are Worker Threads, and when would you use them instead of Child Processes?
3. How would you implement graceful shutdown in a production Node.js service?
4. How would you handle CPU-intensive tasks without blocking the event loop?
5. How would you handle race conditions when multiple requests update the same resource?

LinkedIn — Riddhi Garg, Node.js production interview questions, Sep 2026

1. How would you handle a memory leak in a long-running Node.js service?
2. Explain the Node.js event loop phases and the placement of `setImmediate/process.nextTick`.
3. How would you debug a Node.js API that randomly times out under load?

4. How would you handle graceful shutdown during deployment?
5. How would you design structured logging in a distributed Node.js system?
6. How would you handle backpressure when streaming large files?

LinkedIn — Pooja Singh, Node.js backend interview experience

1. How would you implement rate limiting in a Node.js API?
2. What are Worker Threads and when would you use them over Child Processes?
3. How would you handle file uploads securely in a Node.js server?
4. How do you manage environment variables securely?

AmbitionBox — Cognizant Node.js Backend Developer, 2024

1. What is Promise.all?
2. What is Node.js?
3. What is middleware?

AmbitionBox — Genuin Inc Node.js Backend Developer, 2024

1. How do callbacks, Promises, JWT and sessions work?
2. How would you stop an asynchronous process running in the background?
3. How do closures work?
4. What problem can occur when combining Promises with setTimeout?

AmbitionBox — Persistent Systems Node.js Developer, experienced, 2024

1. What is Node.js and why do we use it?
2. If Node.js is single-threaded, how does it achieve concurrency/multiple-threaded work?

LinkedIn — Vivek Mishra, CTS MERN interview experience, 2026

1. How do Event Loop and Worker Threads relate to Node.js performance?
2. How do Streams, piping and backpressure work?
3. How would you handle large Excel uploads without crashing the server?
4. How would you design a scalable email broadcast service using queues and cloud services?

PART 3 — SENIOR PRODUCTION SCENARIO PRACTICE

Use these after the reported questions. They are synthesized from repeated themes in the interview reports and the uploaded debugging notes, not claimed as verbatim questions from a particular interview.

A. CPU / PERFORMANCE SCENARIOS

1. Production CPU jumps from 35% to 95% after a deployment. What is your first 5-minute investigation?
2. CPU is high on one pod but normal on the other pods. What does that tell you?
3. All pods show a CPU increase immediately after a release. How do you separate traffic growth from a regression?
4. A flame graph shows a huge `JSON.stringify` frame. What would you inspect before changing code?
5. You see many `v8::internal::Heap::*` frames. What hypothesis does that create?
6. A middleware frame is wide in the middle of the stack but has many small children. Where do you investigate?
7. How would you profile a live production process without restarting it?
8. After optimizing a hot function, how would you prove the change actually improved production performance?

B. MEMORY / OOM SCENARIOS

1. RSS grows steadily for 72 hours and the pod eventually OOMs. Walk through your investigation.
2. Heap usage rises and falls like a sawtooth, but the floor keeps rising. What does that suggest?
3. Heap snapshots show `EventEmitter` objects increasing after every request. What do you check?
4. A closure retains a 50 MB request object after the response is sent. How would you find and fix it?
5. An in-memory `Map` grows forever. What production policy would you add?
6. Heap is stable but RSS keeps increasing. What possibilities would you investigate?
7. How do you take heap snapshots safely in production?
8. Why is restarting the process not a complete memory-leak solution?

C. EVENT LOOP / CONCURRENCY

1. API p99 jumps to 8 seconds while host CPU is only 40%. What is your first hypothesis?
2. How would you prove that a synchronous operation is blocking the event loop?
3. A 50 MB JSON document is parsed inside a request handler. What happens to other requests?
4. When would you use Worker Threads instead of simply making a function `async`?
5. Worker Threads vs Child Processes: what are the practical differences for CPU-heavy work?
6. How can `process.nextTick()` starve normal event-loop progress?
7. `setImmediate()` vs `setTimeout(0)`: what changes depending on where they are scheduled?
8. What happens when the libuv thread pool is exhausted?
9. How would you monitor event-loop delay in production?

D. PRODUCTION OPERATIONS

1. How would you investigate a random Node.js API timeout that cannot be reproduced locally?
2. What metrics should you put on a Node.js production dashboard?
3. How would you correlate a latency spike with a specific deployment?
4. How would you safely debug a live process without exposing a debug port to the internet?
5. What would you do if a service crashes with out-of-memory at 2 AM?
6. How would you use a core dump after a crash?
7. How do you decide whether to rollback, hot-fix or continue investigating?
8. How would you design graceful shutdown so in-flight requests are not dropped?

9. How do you handle an unhandled rejection in a production service?
10. How would you investigate a connection-pool exhaustion incident?
11. How would you investigate a queue backlog that keeps growing?
12. How would you diagnose a downstream service becoming slow?

PART 4 — HOW TO ANSWER A PRODUCTION DEBUGGING QUESTION

- 1) Clarify the symptom: CPU, memory, latency, error rate, throughput, one pod or all pods.
- 2) Establish a baseline: current vs previous deploy, p50/p95/p99 latency, CPU, RSS/heap, event-loop delay, traffic.
- 3) Form 2–3 hypotheses; do not jump straight to a fix.
- 4) Pick the lowest-risk evidence source: metrics → logs/traces → profiler/heap snapshot → code/deploy diff.
- 5) Apply a safe mitigation first if users are impacted.
- 6) Find root cause and make the durable fix.
- 7) Verify with the original metric and compare before/after.
- 8) State the trade-off and prevention: alert, regression test, dashboard, limit, timeout, eviction policy, rollout guardrail.

Sources / verification trail

- Uploaded source: Node.js Production Debugging — Full interview-ready notes (5 pages).
- Glassdoor: Laerdal Medical Corp. Bengaluru, Senior MERN Stack Developer interview, 5 Mar 2026.
- LinkedIn: Pravin Pagare, Node.js Interview Experience, September 2026.
- LinkedIn: Ankita Budhia, Full Stack Interview Experience and Preparation Tips, September 2026.
- LinkedIn: Riddhi Garg, Node.js Interview Questions for Production Systems, September 2026.
- LinkedIn: Pooja Singh, Node.js Interview Questions for Backend Developers.
- LinkedIn: Vivek Mishra, CTS MERN Stack interview experience, 2026.
- AmbitionBox: Cognizant Node.js Backend Developer interview questions, updated Nov 2024.
- AmbitionBox: Genuin Inc Node.js Backend Developer interview experience, Mar 2024.
- AmbitionBox: Persistent Systems Node.js Developer experienced interview, Jun 2024.